Jawaban Checkpoint UTS Graf Sains Data

NAMA: **Muhammad Abdillah Caesar** NIM: **23031554245** KELAS: **23D**

PERTANYAAN 1

Apa nama database yang digunakan pada tutorial ini?

Database default yang digunakan dan dikonfigurasi pada kode tutorial tersebut adalah `neo4j` .

Penjelasan Rinci:

- Pada Subbab 1.2 *"Mempelajari Penggunaan Driver Neo4J"*, fungsi pembuat relasi `add_friend` mendefinisikan parameter basis data secara eksplisit:

```
driver.execute_query(
    "MERGE (a:Person {name: $name}) "
    "MERGE (friend:Person {name: $friend_name}) "
    "MERGE (a)-[:KNOWS]->(friend)",
    name=name, friend_name=friend_name, database_="neo4j"
)
```

- Fungsi pembacaan relasi `print_friends` juga mengarahkan kueri ke basis data yang sama menggunakan `database_="neo4j"` dengan mode baca `RoutingControl.READ` .
- Argumen `database_="neo4j"` ini secara pasti mengonfirmasi nama database target pada instance Neo4j lokal. Selain itu, pada bagian Impor Data (Subbab 1.3), juga diperkenalkan database sistem bawaan yaitu `system` .

PERTANYAAN 2

Sebutkan akses lain (seperti sintaks DDL/DML CYPHER) yang dapat digunakan mengakses database lain dalam 1 instance!

Dalam 1 instance DBMS Neo4j (arsitektur *Multi-Database*), berikut adalah sintaks DDL/DML Cypher serta mekanisme Driver API yang dapat digunakan:

1. **Pengubahan Database Target via Driver / Session API:**
   Mengubah parameter `database_` pada fungsi `driver.execute_query(..., database_="nama_db_lain")` atau membuka sesi baru: `session = driver.session(database="nama_db_lain")`
2. **Perintah Data Definition Language (DDL) CYPHER (Eksekusi pada DB `system`):**
   - `CREATE DATABASE nama_db_lain;` — Membuat database baru
   - `DROP DATABASE nama_db_lain IF EXISTS;` — Menghapus database
   - `SHOW DATABASES;` — Menampilkan seluruh database
   - `STOP DATABASE / START DATABASE nama_db_lain;` — Mengubah status database
3. **Perintah Switch Context pada CLI / Cypher Shell:**
   Memindahkan konteks database aktif secara interaktif: `:use nama_db_lain`
4. **Composite Databases & Fabric (Cross-Database Query / DML):**
   Membuat kueri lintas database dalam 1 instance menggunakan klausa `USE`:

PERTANYAAN 3

Apa jenis graf pertemanan yang telah dibuat di sini?

Jenis graf pertemanan yang telah dibuat pada tutorial tersebut adalah **Graf Berarah (Directed Graph / Digraph) yang Berlabel (Labeled) dan Berproperti (Property Graph)**.

1. Sifat Directed (Berarah)
Orientasi busur memiliki arah spesifik dari simpul asal ke simpul tujuan.

2. Sifat Labeled
Memiliki label simpul (`:Person`) dan tipe relasi (`:KNOWS`).

3. Sifat Property Graph
Node menyimpan atribut kustom berbentuk key-value (properti `name`).

4. Sifat Simple Graph
Bebas dari relasi ganda per arah berkat penggunaan klausa `MERGE` .

PENJELASAN RINCI DAN RUNUT:

- **Sifat Directed (Berarah):**
  Pada fungsi `add_friend` , perintah Cypher yang digunakan untuk membentuk relasi adalah:

  ```
  MERGE (a)-[:KNOWS]->(friend)
  ```

  Tanda panah `-[:KNOWS]->` secara spesifik dan eksplisit menetapkan arah relasi dari simpul `a` menuju simpul `friend` . Artinya, jika Arthur memiliki relasi `KNOWS` ke Guinevere (`(Arthur)-[:KNOWS]->(Guinevere)`), relasi tersebut bersifat satu arah dan tidak otomatis berlaku sebaliknya kecuali dibuat relasi baru secara terpisah.
- **Sifat Labeled (Memiliki Label & Tipe):**
  Setiap simpul yang dibentuk dikategorikan ke dalam label `:Person` , sedangkan hubungan antar simpul diberi klasifikasi tipe relasi `:KNOWS` .
- **Sifat Property Graph:**
  Simpul menyimpan atribut data kontekstual berbentuk pasangan kunci-nilai (key-value properties), yaitu properti `name` pada setiap node `Person` .
- **Sifat Simple Graph (Unweighted Unigraph):**
  Penggunaan klausa `MERGE` memastikan tidak ada relasi ganda (duplicate edge) dengan tipe dan arah yang sama di antara dua simpul yang sama, sehingga graf terjamin bersifat *Simple Directed Graph*.

LAPORAN HASIL RUNNING & METODE MATEMATIS SOAL 2.1 & 2.2

Verifikasi Presisi Berdasarkan Sumber Referensi Eksekusi (hasil.pdf) — E-Commerce RetailRocket & Neo4j

BAGIAN 2.1: WRANGLING DAN EKSPLORASI DATA GRAF (EDA)

1. Filter Periode Waktu & Pembuatan File CSV (Soal 2.1.1 & 2.1.2)

Penyaringan log interaksi e-commerce **events.csv** (total 2.756.101 baris) menjadi dua rentang periode waktu:

Periode Fold	Rentang Tanggal	Jumlah Data	File Tersimpan
Fold Training (P1)	2015-05-01 s.d. 2015-06-30	1.179.597 baris	events_P1_train.csv
Fold Testing (P2)	2015-07-01 s.d. 2015-08-31	1.232.719 baris	events_P2_test.csv

2. Komposisi Subgraph & CYPHER SQL Query (Soal 2.1.3)

Sebanyak 1.000 sampel data P1 disuntikkan ke Neo4j. Query Cypher untuk mengekstraksi komposisi label simpul:

```
MATCH (n)
RETURN labels(n) AS JenisSimpul, count(n) AS Jumlah
```

Uraian CYPHER: (1) MATCH (n): Memindai seluruh node; (2) labels(n): Mengambil label simpul (Visitor/Item); (3) count(n): Mengagregasi total simpul tiap label.
Output Komposisi Subgraph: Simpul **Visitor = 834 unit** | Simpul **Item = 919 unit**.

3. Karakteristik Vector Space Modelling (VSM) User & Item (Soal 2.1.4 & 2.1.5)

Pemodelan ruang vektor menggunakan pembobotan **TF-IDF**: $TF\text{-}IDF(t,d,D) = TF(t,d) \times log(|D| / (1 + |\{d \in D : t \in d\}|))$.

Entitas VSM	Bentuk Matriks (Shape)	Dimensi Fitur	Entri Non-Zero / Sparsity
VSM User (Fitur Item)	(596.191, 161.411)	161.411 Unique Items	909.454 (Sparsity: 100.00%)
VSM Item (Fitur Visitor)	(161.413, 596.189)	596.189 Unique Visitors	909.464 (Sparsity: 100.00%)

4. Analisis Proyeksi Graf (Efektivitas vs Efisiensi) (Soal 2.1.6)

• **Efisiensi Komputasi (Proyeksi Item-Item):** Katalog produk relatif terikat dibanding jumlah pengunjung yang tumbuh dinamis. Matriks Item-Item lebih hemat memori.
• **Efektivitas Rekomendasi (Proyeksi User-User):** Proyeksi User-User secara eksplisit merepresentasikan kesamaan perilaku (*user preference similarity*) untuk personalisasi.
• **Kesimpulan:** Proyeksi Item-Item unggul dalam efisiensi komputasi, Proyeksi User-User unggul dalam efektivitas personalisasi.

5. Similarity & Output CYPHER SQL Perubahan Graf (Soal 2.1.7 & 2.1.8)

Dihitung menggunakan Cosine Similarity: $Sim(u■, u■) = (u■ \cdot u■) / (||u■||■ \times ||u■||■)$.
Output Top 5 Similarity Record (Neo4j Cypher Output): 1. User 58 ↔ User 686 (1.0000) | 2. User 572 ↔ User 934 (1.0000) | 3. User 753 ↔ User 814 (1.0000) | 4. User 466 ↔ User 949 (1.0000) | 5. User 268 ↔ User 728 (0.7204)

Kondisi Graf	CYPHER SQL Query	Contoh Hasil Output Cypher

SEBELUM
(Bipartite User-Item)
	`MATCH (v:Visitor)-[r]->(i:Item)`
	`WHERE type(r) <> 'SIMILAR'`
	`RETURN v.id, type(r), i.id LIMIT 5`
		Visitor (257597) -[:VIEWED]-> Item (355908)
		Visitor (992329) -[:VIEWED]-> Item (248676)
		Visitor (111016) -[:VIEWED]-> Item (318965)

SESUDAH
(Terproyeksi User-User)
	`MATCH (v1:Visitor)-[s:SIMILAR]->(v2:Visitor)`
	`RETURN v1.id, type(s), v2.id, s.score`
	`ORDER BY s.score DESC LIMIT 5`
		User (58) -[:SIMILAR {score: 1.0000}]-> User (686)
		User (572) -[:SIMILAR {score: 1.0000}]-> User (934)
		User (753) -[:SIMILAR {score: 1.0000}]-> User (814)

BAGIAN 2.2: ANALYTICS (REKOMENDASI KNN & EVALUASI METRIK)

1. Pemilihan User Acak & K-Nearest Neighbors (k=5) (Soal 2.2.1)

• **Skema Pengacakan:** Ditentukan dengan *Fixed Seed* np.random.seed(2026) pada subset 28.747 user aktif di P1 & P2.
• **User Terpilih: Visitor ID 113265**.
• **3 Item Diiklankan (k=5 KNN):** ['200425', '164941', '139394'].

Peringkat	Visitor ID Tetangga (KNN)	Skor Similarity	Keterangan
1	Visitor 113265	1.0000	Self Target User
2	Visitor 600755	1.0000	Identical Interacted Items
3	Visitor 995624	1.0000	Identical Interacted Items
4	Visitor 909839	1.0000	Identical Interacted Items
5	Visitor 469324	0.0000	Non-overlapping Neighbor

2. Perhitungan Metrik Evaluasi Hit@5 & NDCG@5 (Soal 2.2.2)

Ground Truth P2 (Testing) Visitor 113265: ['235154']
Daftar Top-5 Rekomendasi Evaluasi: ['200425', '164941', '139394', '47006']

Metrik Evaluasi	Formula / Rumus Matematis	Hasil Skor
Hit@5	`Hit@K = 1.0 jika |Rekomendasi_K ∩ GroundTruth_P2| > 0, else 0.0`	**0.0000**
NDCG@5	`DCG@K = Σ (rel_i / log2(i+1)), NDCG@K = DCG@K / IDCG@K`	**0.0000**

Catatan & Analisis Evaluasi: Nilai Hit@5 dan NDCG@5 sebesar **0.0000** menunjukkan tingginya *data sparsity* pada log transaksi e-commerce, di mana item yang dilihat/dilihat ulang pengunjung pada periode testing (P2) bersifat sangat dinamis dan mengalami *cold-start item / sifting behavior* dibanding histori interaksi P1.